# Lux: Current Architecture Specification

Visual Output Surface for AI Agents — a System in Transition

July 2026 — v0.21.0

## Contents

# 1 Overview

Lux is a visual output surface for Claude Code agents and local applications. An agent or an app describes a user interface as a tree of typed JSON elements and submits it to Lux; Lux installs that interface in one authoritative store and renders it in a native ImGui window.

The system is built from one engine fronted by thin client surfaces. The engine is split into two cooperating processes:

- The **Hub** (`luxd`) owns the authoritative interface state, runs the real interaction handlers, and coordinates the display. It is the single front door: every client talks to the Hub.
- The **Display** (`lux-display`) holds a full replica of the interface and renders it every frame. It captures user input and forwards each interaction back to the Hub. It never talks to a client directly.

Four client surfaces reach the Hub, and each is thin:

- The **library** surface is the operations layer itself — a direct in-process call into the engine.
- The **MCP** surface is a set of tools an agent calls; it speaks streamable HTTP to the Hub.
- The **REST** surface is a typed FastAPI application on the Hub; it is the front door for the command-line tool and any non-MCP caller.
- The **CLI** (`lux`) is an HTTP client of the REST surface.

## 1.1 Current and Target

This document describes the architecture *as the code stands today*. The canonical design intent lives in `docs/architecture/target/target.md`; where this document and that target disagree, the target defines the *direction* and this document reports the *present*. Each place the present diverges from the target is called out explicitly.

The code has converged substantially on the target. The Hub is authoritative, the operations layer is the single home of every capability, the MCP tools and REST routes are adapters over it, and the command-line tool reaches the Hub over REST. Two areas remain in transition:

- **Element migration.** Fifteen of the twenty-five element kinds are on the Element-ABC path (data and behavior on the class); ten kinds are still frozen dataclasses decoded by per-kind codecs. See Section §3.
- **The display's own state.** The Display still owns its render loop, its themes and window settings, and its diagnostic ring buffers. The Hub reaches those facts by proxying a read or write over its one connection to the Display (Section §7).

## 1.2 Companion Documents

The sibling files provide narrower supporting views:

- `target/target.md` — the canonical rewrite target. Start there for design intent; return here for current behavior.
- `one-code-path.md` — the design of the operations layer, the REST front door, the thin adapters, and the streamable-HTTP transport. It is the design record for most of what this document now describes as shipped (ADR DES-055).
- `target/introspection-api.md` — the verification and control surface an agent uses to inspect live Hub and Display state.

Four formal models hold parts of the system to a checked specification; they are cross-referenced where each applies and listed together in Section §11.

### 1.3 Design Principles

**P1**. **One engine, thin clients.** Every capability is one typed operation on the Hub. The library call, the MCP tool, the REST route, and the CLI command all run that same operation; none reimplements it.

**P2**. **The Hub is the authority.** Interface state that a client submits is owned by the Hub. The Display is a replica, and it loses every disagreement.

**P3**. **Protocol is the API surface.** An agent describes an interface as JSON; Lux renders it. There is no shared memory and no required Python import.

**P4**. **UI state crosses the wire; render calls do not.** Serialized elements, scene updates, and interaction messages travel between Hub and Display. ImGui calls stay inside the Display.

**P5**. **Whole-UI resend on change.** When Hub-side state changes, the Hub re-sends the whole affected interface and the Display replaces its copy. There is no incremental diff protocol.

## 2 System Architecture

### 2.1 Component Map

The package divides into the engine and its client surfaces. The engine is the operations layer, the Hub domain, the scene state, and the protocol; the surfaces are the MCP tools, the REST routers, the CLI, and the transport plumbing that carries each. The Display is the engine's second process.

| Concern | File or package | Responsibility |
|---|---|---|
| Operations | `src/punt_lux/operations/` | The engine's front of house. One `Operations` facade composes concern classes — `SceneOperations`, `QueryOperations`, `MenuOperations`, `DisplayControlOperations`, `PubSubOperations`, `ConvenienceOperations`, `DisplayModeOperations`. Each method takes a typed request and returns a typed result or an `OpError`. Models live in `src/punt_lux/operations/models/`. |
| Hub domain | `src/punt_lux/domain/hub/` | The authoritative engine core. `HubDisplay` is the store; `HubReplicator` is the sole writer to the Display; `FrameLifecycle`, `FrameExpiry`, and `ExpirySweep` own the frame time-to-live; `HubReads` answers introspection; `HubMenuRegistry` owns the menu bar; `HubSceneWriter` applies updates and clears; `ClientRegistry` owns luxd's one display connection and the D21 dispatch. |
| Domain (pure) | `src/punt_lux/domain/` | The renderer-free and socket-free element model. `element_abc.py` is the `Element` ABC (data and behavior); `interaction.py` and `container_interaction.py` are the typed events; `handlers/remote_dispatch.py` wraps handlers for the two-tier path. |
| Scene | `src/punt_lux/scene/` | The Display's scene graph. `SceneManager` owns the unframed scenes and per-scene widget state; `FrameBook` owns the frames and the scene-to-frame and scene-to-owner maps; `frame.py` is the `Frame` window. |
| Protocol | `src/punt_lux/protocol/` | The wire contract: length-prefixed JSON framing, the `FrameReader` streaming buffer, the twenty-five element kinds in `src/punt_lux/protocol/elements/`, and the message families in `src/punt_lux/protocol/messages/`. |

| Concern | File or package | Responsibility |
|---|---|---|
| Display | src/punt_lux/display/ | The ImGui render loop and coordinator. server.py, element_renderer.py, table_renderer.py, menu_manager.py, idle_screen.py, and texture_cache.py. |
| MCP tools | src/punt_lux/tools/ | Thin adapters over the operations facade. tools.py and composite_tools.py are display-facing tools; subscribe_tools.py are the session pub-sub tools; server.py owns the FastMCP instance. |
| REST | src/punt_lux/rest/ | The typed FastAPI surface. RestSurface composes concern routers (scenes, menus, display, config); status.py is the one OpError-to-HTTP table. |
| MCP transport | src/punt_lux/mcp_transport.py | luxd's MCP leg: streamable HTTP mounted at /mcp beside the REST routes. mcp_endpoint.py resolves session identity; mcp_session.py runs the per-session cleanup cascade; session_cleanup.py tears down a departed session's Hub state. |
| CLI transport | src/punt_lux/rest_client.py | LuxRestClient — the CLI's typed HTTP client of luxd's REST routes. rest_transport.py is the HTTP transport; rest_loopback.py is the in-process transport tests use. |
| Gateway | src/punt_lux/luxd.py | Builds the one FastAPI app on uvicorn, mounts the MCP leg and the REST surface, and starts the frame-expiry sweep on the event loop. |
| Display client | src/punt_lux/display_client.py | DisplayClient — the Unix-socket client of the Display. Constructed in exactly one place, src/punt_lux/domain/hub/clients.py, and an AST guard enforces that (Section §6). |
| Paths | src/punt_lux/paths.py | DisplayPaths — display socket discovery, PID lifecycle, and auto-spawn. src/punt_lux/hub_paths.py is the symmetric HubPaths for luxd's port and PID files. |
| Apps/Beads | src/punt_lux/apps/beads.py | BeadsBrowser — builds the beads issue table as an element tree. Pure: it holds no socket. |
| Config | src/punt_lux/config.py | ConfigManager — reads and writes the per-repo display-mode toggle in <repo>/.punt-labs/lux.md. |
| CLI | src/punt_lux/__main__.py | The Typer app: show, service, status, and installation commands. |

## 2.2 The Operations Facade

Every capability the system offers is one method on the Operations facade (src/punt_lux/operations/facade.py). The facade composes seven concern classes, each owning one group of capabilities, and delegates each method to the right one. A caller — an MCP adapter, a REST route, or a test — holds one object and calls one method.

The facade binds no process singletons at import time. Its for_store classmethod receives every collaborator (the HubDisplay store, the replicator, the Hub, the client registry, the menu registry, the display connection) from the composition root, so the operations are testable without the running process.

### 2.2.1 Typed Requests and the Never-Raising Parse

Each operation takes a typed request model and returns either the operation's own success model or a shared error. Validation is the operation's job, not the adapter's. A request model carries a never-raising parse classmethod: it validates raw arguments and returns either the request or an OpError, so an adapter that builds a plain mapping and calls parse can never raise past its string contract. The operation accepts the parse result and passes an OpError straight through.

The consequence is one schema per capability, applied at one point. A bad layout becomes an invalid_request error inside parse; the MCP adapter renders it to the legacy status string, and a

REST route renders it to an HTTP 422. The rules live in one place; only the rendering differs by surface.

#### 2.2.2 The Error Contract

Every operation may return `OpError` (`src/punt_lux/operations/models/common.py`), a frozen model tagged `kind="error"` so a result cannot be both a success and a failure. The `code` is a closed set the caller branches on; the `reason` is the human sentence for logs and messages.

| Code | Meaning |
|---|---|
| `display_unavailable` | The display process is not running. |
| `timeout` | A proxied round-trip to the display exceeded its bound. |
| `rejected` | The engine refused the caller's request (the caller's fault). |
| `invalid_request` | The request itself did not type-check. |
| `not_found` | The named scene or resource does not exist. |
| `fault` | An engine-side failure: a malformed display reply, an unreadable config file. |

The REST surface maps each code to one HTTP status through a single shared table (`src/punt_lux/rest/status.py`): `invalid_request` to 422, `not_found` to 404, `rejected` to 409, `fault` to 502, `display_unavailable` to 503, and `timeout` to 504. The mapping is a table, not per-route logic, so a new operation inherits it for free. `LuxRestClient` reads the same table in reverse, so the CLI observes the same contract from the other end.

### 2.3 Data Flow

Three data paths connect a client to the Display.

#### 2.3.1 Write Path (Client → Hub → Display)

1. A client calls a capability — the `show` MCP tool, a `PUT /scenes/{id}` REST call, or `lux show beads`.
2. The surface builds a plain mapping and calls the request builder, then calls one operation. For `show` that is `Operations.render`.
3. The operation validates the request, then installs the scene into `HubDisplay` — the authoritative store — under the store lock, indexing every element and its owner.
4. The operation marks the scene dirty on the `HubReplicator` and returns its typed result at once. The client's call does not wait on the Display.
5. The `HubReplicator`, the one background worker that writes to the Display, drains the dirty set, serializes the scene, and sends it over luxd's single socket connection.
6. The Display receives the scene, replaces its replica, and renders it every frame.

#### 2.3.2 Event Path (User → Hub, two-tier D21)

The event path is where the Hub/Display split is clearest. The Display does not run interaction handlers; it forwards each interaction to the Hub, where the real handler runs.

1. When the Display installs a scene, it wraps every handler on each element, replacing each event bucket with a `RemoteDispatchGroup` that captures the socket-send closure.
2. The user interacts with a widget. The renderer fires the typed event on the Display's element copy, and the wrapped handler runs instead of the real handler.
3. The wrapper builds one `RemoteEventHandlerInvocation` (`element_id`, `action`, `event_kind`, `value`, `ts`, `scene_id`) and sends it to the Hub. One interaction produces one message per element/event bucket, however many handlers that bucket holds.

4. The Hub receives the message in `ClientRegistry._hub_interaction_dispatch`, resolves the authoritative element by `(scene_id, element_id)`, checks its owner, builds the typed event, and fires the real handler chain once on its own copy.

5. If the handler mutated Hub state (for example a dialog dismissed itself), the Hub marks the scene dirty; the replicator re-sends the whole scene and the Display renders the current tree.

A `frame_close` action is not an element interaction: the Display sends it when the user closes a frame, and the Hub answers by removing that frame's scenes so both tiers agree the frame is gone (Section §5).

#### 2.3.3 Update Path (Incremental Patching)

1. A client calls `update(scene_id, patches)`, reaching `Operations.update`.

2. The operation applies the patch batch to `HubDisplay` through `HubSceneWriter`: a `set` patch changes fields on the target element, a `remove` patch drops it.

3. The operation marks the scene dirty; the replicator re-sends the whole scene, and the Display renders the result.

# 3 The Hub/Display Slice

This section describes the Hub-authoritative core and the Element-ABC path the migration is converging on. It is where new element work lands and the reference for how the whole system is meant to behave once migration completes.

The domain code in `src/punt_lux/domain/` imports no ImGui, no socket, and no JSON. Adapters wrap it: the Display renders it, and the replicator moves its serialized elements over the wire.

## 3.1 HubDisplay: the Authoritative Store

`HubDisplay` (`src/punt_lux/domain/hub/hub\_display.py`) is the Hub's single authoritative surface for scene state. It is a facade over typed collaborators, each with one responsibility:

| Collaborator | Responsibility |
|---|---|
| `ElementIndex` | `(scene_id, element_id)` → `Element` lookup. |
| `OwnerTracker` | Every element's owning connection; the interaction path checks it, and the disconnect path walks it. |
| `RootRegistry` | Scene-root elements that take part in the property-observer cascade. |
| `ChildIndex` | Parent-to-children edges captured at install time so cascade removal drops every descendant in one walk. |
| `HubClientRegistry` | The connections currently registered as Hub clients. |
| `FrameLifecycle` | Each scene's frame presentation, its optional TTL deadline, and its teardown — all under the store lock (Section §5). |
| `HubReads` | The authoritative introspection reads that answer `inspect_scene` and `list_scenes`. |

A write dispatches on the typed `Update` sum — `AddElement`, `SetProperty`, `RemoveElement` — and enforces ownership: a connection cannot mutate or evict another connection's elements. `AddElement` recurses into composite children so a button buried inside a dialog is indexed and later click-resolvable. Re-showing a scene removes the connection's previous roots and installs the new ones through the same write path, so ownership, root observers, and child indexes rebuild in one place.

## 3.2 The Element ABC

The migrated kinds subclass the `Element` ABC in `src/punt_lux/domain/element\_abc.py`. The ABC carries data *and* behavior:

- **Composite render template.** `render()` is a template method and is never overridden; a composite overrides `_children()` to return its children, and a leaf inherits the empty default.
- **Handler registry.** `add_handler`, `remove_handler`, and `fire` key handlers by `Event` subclass. `fire` snapshots the bucket so a handler that mutates the registry mid-dispatch cannot affect the in-flight call.
- **Property-observer surface.** `add_observer`, `mark_removed`, and `removed` let a parent composite react to a child being removed. All three removal paths — an agent `RemoveElement`, a component self-dismiss, and a connection disconnect — route through the one `mark_removed` method.
- **Native serialization.** `__reduce__` and `__setstate__` let an element cross the Hub-to-Display boundary. Hub-only bookkeeping (the observer closures that reference `HubDisplay`) is dropped on serialization; the Display is a replica and does not need Hub observers. Handlers survive so the Display can wrap them for remote dispatch.

## 3.3 Two-Tier Handler Dispatch (D21)

The Hub says: "I am sending you a copy of an interface. If anyone interacts with it, I will do the work." The Display says: "I will render that copy and forward interactions back to the Hub."

Both tiers hold the same element state and both call `elem.fire(event)`. The difference is what the handler does:

- **Hub:** the real handler runs — `call_model`, `publish`, `mark_removed`.
- **Display:** the wrapped handler sends one `RemoteEventHandlerInvocation` to the Hub, which resolves the authoritative element, checks its owner, and fires the real event once.

Four event kinds cross this boundary today:

| Event | Wire kind | Fired by |
|---|---|---|
| `ButtonClicked` | `button_clicked` | a button press. |
| `ValueChanged` | `value_changed` | a checkbox, slider, combo, radio, selectable, or text-input change. |
| `HeaderToggled` | `header_toggled` | a `collapsing_header` opening or closing. |
| `TabChanged` | `tab_changed` | a `tab_bar` switching tab. |

`ButtonClicked` and `ValueChanged` are in `src/punt_lux/domain/interaction.py`; `HeaderToggled` and `TabChanged` are in `src/punt_lux/domain/container\_interaction.py`. The Hub builds the typed event from the wire kind and value, denying by default when the value has the wrong shape. Grouping is per element/event bucket: a button with three handlers still produces one remote message, and the Hub replays the full chain once.

## 3.4 Agent Subscribe / Publish

The Hub also owns an application-level publish/subscribe channel (`src/punt_lux/domain/hub/hub.py`), distinct from the element-observer mechanism above; the two share no machinery.

Every operation is scoped to a connection. A connection cannot see, touch, or publish into another connection's topics. `subscribe` registers the caller's outbound writer against a topic; `publish` fans an `ObserverMessage` payload out to that connection's subscribers using snapshot-then-iterate, so a slow handler cannot stall concurrent publishes. Topics — `openTicket`, `closeTicket`, `markTicketInProgress` — are defined by the app or agent, not by Lux. This is the business-event channel, not the interface-replication path.

### 3.5 Migrated vs. Still-Legacy

| State | What it covers |
|---|---|
| **Migrated** | The fifteen Element-ABC kinds: `text`, `button`, `checkbox`, `dialog`, `progress`, `input_text`, `input_number`, `slider`, `color_picker`, `combo`, `radio`, `selectable`, `group`, `collapsing_header`, and `tab_bar`. Three of these — `group`, `collapsing_header`, `tab_bar` — are ABC containers, migrated only when their whole subtree is ABC. `HubDisplay` authority with ownership and cascade removal, the D21 four-event path, the frame lifecycle, and Agent Subscribe/Publish are all in this column. |
| **Transitional** | The Display process, which runs the D21 event path yet still owns rendering, scene tabs, frames, and its own diagnostic buffers as in the legacy design. |
| **Legacy** | The ten not-yet-migrated element kinds as frozen dataclasses: `markdown`, `image`, `separator`, `spinner`, `draw`, `plot`, `table`, `tree`, `window`, and `modal`. |

The direction is fixed by `target/target.md`; the pace is set by migrating one element family at a time, container plus primitives first (the migration plan, DES-041).

# 4 Wire Protocol

## 4.1 Framing

Every message is a length-prefixed JSON frame:

```
+-----------------+---------------------------+
| 4 bytes (uint32) | N bytes (UTF-8 JSON) |
| big-endian | payload |
+-----------------+---------------------------+
```

Maximum payload size is 16 MiB ($2^{24}$ bytes). The `FrameReader` (in `src/punt_lux/protocol/\_\_init\_\_.py`) accumulates partial reads into a byte buffer and yields complete frames via `drain()` (raw dicts) or `drain_typed()` (Message dataclasses). An oversize frame raises `ValueError`, which the socket layer converts to a client disconnect.

This framing is the Hub-to-Display wire. A client reaches the Hub over HTTP (MCP streamable HTTP, or REST), not over this socket; the socket is Hub-internal (Section §6).

## 4.2 Message Types

The message layer is split across six families in `src/punt_lux/protocol/messages/`: `scene`, `lifecycle`, `menu`, `introspect`, `observer`, and `remote_invocation`. There is no standalone `interaction` family — a user interaction travels as `RemoteEventHandlerInvocation` from the `remote_invocation` module. A `MessageRegistry` (`src/punt_lux/protocol/messages/registry.py`) is the wire dispatch table, populated at import time by each family module; a test can build an isolated registry without touching the module default.

#### 4.2.1 Client → Display

| Type | Purpose |
|---|---|
| `SceneMessage` | Replace or add a scene. Fields: `id`, `elements[]`, `title`, `layout`, `frame_id`, `frame_title`, `frame_size`, `frame_flags`, `frame_layout`. |
| `UpdateMessage` | Incremental patches. Fields: `scene_id`, `patches[]` (each: `id`, `set`, `remove`). |
| `ClearMessage` | Remove scenes and reset state. |
| `PingMessage` | Heartbeat. Field: `ts`. |
| `ConnectMessage` | Client identity handshake. Field: `name`. |
| `MenuMessage` | Set the menu bar. |
| `RegisterMenuMessage` | Register per-connection menu items. |
| `ThemeMessage` | Apply a theme by name. |
| `QueryRequest` | A control or introspection request the display answers. Fields: `method`, `params`. |

#### 4.2.2 Display → Client

| Type | Purpose |
|---|---|
| `ReadyMessage` | Handshake on connect: protocol version and capabilities. |
| `AckMessage` | Acknowledges a scene or update: `scene_id`, `ts`, optional `error`. |
| `RemoteEventHandlerInvocation` | A user interaction forwarded to the owning Hub: `element_id`, `action`, `event_kind` (`button_clicked`, `value_changed`, `header_toggled`, `tab_changed`), `value`, `ts`, optional `scene_id`. |
| `ObserverMessage` | A connection-scoped business event for the Agent Subscribe/Publish path: `topic` + `payload`. |
| `PongMessage` | Ping response. |
| `QueryResponse` | A control or introspection response: `method`, `result`, optional `error`. |

An unknown message type deserializes as `UnknownMessage` for forward compatibility rather than disconnecting the peer.

### 4.3 Element Kinds

The wire layer exposes twenty-five element kinds plus the non-element `Patch` payload used by `UpdateMessage`. Fifteen kinds are on the Element-ABC path (Section §3); the remaining ten are frozen, slotted dataclasses registered into `ElementCodec`. The `Element` union is assembled in `src/punt_lux/protocol/elements/\_\_init\_\_.py`.

An ABC kind serializes two ways. Its Level-1 form is a plain JSON dict through its per-kind codec; its wire form crossing the Hub-to-Display boundary is a base64-encoded pickled entry inside the `SceneMessage` codec. A legacy kind crosses as a plain dict either way.

| Category | Element | Notes |
|---|---|---|
| `basics` | `text` | Content plus style (heading, caption, success, error, code); optional `color` hex string. ABC. |
| | `markdown` | CommonMark via imgui_md. Legacy. |
| | `image` | File path or base64 data. Legacy. |
| | `separator` | Visual divider. Legacy. |
| | `progress` | Fraction bar with label. ABC. |
| | `spinner` | Loading indicator. Legacy. |
| `inputs` | `button` | Label, action, disabled; `arrow` and `small` variants. ABC. |

| Category | Element | Notes |
|---|---|---|
| | slider | Min, max, value, integer mode. ABC. |
| | checkbox | Boolean toggle; fires ValueChanged. ABC. |
| | combo | Dropdown selection. ABC. |
| | input_text | Single-line text input. ABC. |
| | radio | Radio button group. ABC. |
| | input_number | Numeric input with step buttons and clamping. ABC. |
| | color_picker | RGB/RGBA hex picker. ABC. |
| | selectable | Click-to-select item; fires ValueChanged. ABC. |
| dialog | dialog | Dialog root with title, children, handlers, and a model-backed lifecycle. ABC. |
| layout | group | Row, column, or paged layout. ABC container. |
| | collapsing_header | Expandable section; fires HeaderToggled. ABC container. |
| | tab_bar | Tabbed container; fires TabChanged. ABC container. |
| | window | Movable, resizable floating panel. Legacy. |
| | tree | Recursive node hierarchy; flat flag. Legacy. |
| | modal | Modal popup; emits "closed". Legacy. |
| graphics | draw | Canvas with typed draw commands (§4.4). Legacy. |
| | plot | ImPlot: line, scatter, bar series. Legacy. |
| table | table | Columns, rows, built-in filters, detail panel, pagination. Legacy. |

## 4.4 Draw Commands

The draw element holds a tuple of typed DrawCommand records. Each command is a frozen, slotted dataclass that satisfies the DrawCommand Protocol (TYPE: ClassVar[str], to_wire(self) -> dict, from_wire(cls, d, *, ctx) -> Self).

| Module | Record | Geometry |
|---|---|---|
| draw_commands_line | Line | Start, end, color, thickness. |
| | Polyline | Point list, color, thickness, closed flag. |
| draw_commands_shape | Rect | Top-left, bottom-right, color, thickness, rounding, filled. |
| | Circle | Center, radius, color, thickness, filled. |
| | Triangle | Three points, color, thickness, filled. |
| draw_commands_curve | BezierCubic | Four control points, color, thickness. |
| draw_commands_text | TextGlyph | Position, content, color. |

Shared value classes live in draw_values.py and draw_bounds.py: Point2 (a validated $[x, y]$ pair), Color (#RRGGBB / #RRGGBBAA hex), Thickness (a strictly positive stroke width), Radius (strictly positive), and Rounding (non-negative). The WireContext in draw_wire.py carries the per-error prefix and the require-or-raise helpers. DrawCommandDecoder in draw_decoder.py dispatches on the wire cmd string

to the right `from_wire` classmethod; `DrawElement.commands` is typed `tuple[DrawCommand, ...]` and the renderer dispatches via a typed `match` statement.

# 5 Scene and Frame Lifecycle

A scene targets a named *frame* — a persistent ImGui window that organizes the workspace. The lifecycle answers three questions the same way on both tiers: when a frame appears, when it disappears, and how long it lives without a refresh.

## 5.1 Frames Die With Their Scenes

A frame exists to show scenes. When a frame's last scene is removed, the frame closes; a frame still holding other scenes stays open.

The Hub blanks a scene by pushing it with no elements. The Display treats an empty-element push as a removal, not as an empty window: an unframed empty push dismisses the scene, and a framed one dismisses the scene from its frame and closes the frame once it holds nothing. This is the amended contract the legacy display-server model now checks (Section §11).

When the user closes a frame on the Display, the Display sends a `frame_close` action. The Hub removes every scene that frame held from `HubDisplay` and marks each dirty, so the authoritative store agrees with the window the user already closed. The blank the replicator then sends is idempotent with that close.

## 5.2 Scenes Survive Their Session

A scene outlives the connection that created it. When an MCP session ends — the client disconnects, or the SDK reaps it on idle — the Hub forgets that connection as a client and releases its subscriptions and inbox, but it leaves the scenes installed. The scenes stay owned by the departed connection id, so a later explicit removal — a frame close, a `clear`, or a TTL expiry — can still reference the owner. Nothing is blanked, so no repaint is marked. A session's death does not erase what it drew.

## 5.3 Frame Time-to-Live

A frame may carry a time-to-live. Shown with a `ttl_seconds`, it is removed once that many seconds pass with no re-show refreshing it. Three small components implement this, all in `src/punt_lux/domain/hub/`:

- `FrameExpiry` owns the bookkeeping: a `frame_id` to a deadline on a monotonic clock (a duration, immune to wall-clock adjustment). It answers which frames are due now and how long until the soonest one. It holds no lock.
- `FrameLifecycle` owns each scene's frame presentation, its deadline, and its teardown, all under the one store lock. Recording a presentation and arming its deadline is a single locked step, so a re-show that refreshes a frame cannot install it yet forget to re-arm it.
- `ExpirySweep` is the task on luxd's event loop that enforces the deadlines. It sleeps until the soonest deadline (capped at a one-second idle poll), sweeps the frames now due, and marks their scenes dirty so the replicator blanks both tiers — the same path a manual frame close takes. It is one more store mutator on the loop, not a second timer thread.

Because arming a deadline and expiring a frame both run under the store lock, a frame re-armed with a fresh TTL is never torn down by a stale deadline: the re-show and the sweep serialize, and whichever runs second sees the other's effect. A re-show carrying no TTL makes the frame permanent. This atomicity is the safety property the frame-expiry formal model checks (Section §11).

A due frame is peeked, not consumed: the sweep tears each frame down and only then disarms its deadline, so a tear-down that raises leaves the frame armed to retry on the next sweep rather than stranded — torn down on the Hub but never blanked on the Display. Frames are isolated from one another: a frame whose tear-down raises is logged and skipped, and every frame that did tear down is still repainted.

### 5.4 The Frame Window

The `Frame` class (`src/punt_lux/scene/frame.py`) is the Display's window for a set of scenes. `FrameBook` (`src/punt_lux/scene/frame\_book.py`) owns the frame collection and the scene-to-frame and scene-to-owner maps, split out of `SceneManager` so each has one job.

| Field | Type and Purpose |
|---|---|
| `frame_id` | `str` — unique identifier. |
| `title` | `str` — window title bar. |
| `scenes` | `dict[str, SceneMessage]` — the frame's scenes, keyed by `scene_id`. |
| `scene_order` | `list[str]` — tab ordering. |
| `active_tab` | `str \| None` — the visible scene. |
| `minimized` | `bool` — docked to the bottom bar. |
| `cascade_index` | `int` — stagger offset for initial placement. |
| `initial_size` | `tuple[int,int] \| None` — first-use size hint. |
| `flags` | ImGui window flags (`no_resize`, `no_collapse`, `no_title_bar`, `no_background`, `no_scrollbar`, `auto_resize`). |
| `layout` | `Literal["tab", "stack"]` — multi-scene composition mode. |

Left-clicking the docking background opens the World menu: list frames, restore minimized frames, close all frames, and reach display settings. A minimized frame appears as a button in a bottom dock bar; clicking it restores the frame.

## 6 Transport and Process

luxd runs one FastAPI application on one uvicorn port (default 8430). Two surfaces mount on that app: the MCP leg at `/mcp`, and the typed REST routers beside it. A `/health` route reports liveness and the live session count.

### 6.1 The MCP Leg: Streamable HTTP

luxd's MCP surface is streamable HTTP (`src/punt_lux/mcp\_transport.py`), the MCP SDK's current transport, mounted as an ASGI application at `/mcp` on the same FastAPI app as the REST routes. There is no separate WebSocket endpoint, and mcp-proxy no longer sits in lux's path: Claude Code connects to `http://127.0.0.1:8430/mcp` directly through its native HTTP MCP configuration.

Each MCP session has its own identity. `McpAsgiApp` (`src/punt_lux/mcp\_endpoint.py`) resolves the session identity on each request and refuses the reserved REST connection key. `SessionScopedServer` (`src/punt_lux/mcp\_session.py`) runs the Hub cleanup cascade when a session ends, and `session_cleanup.py` tears down that session's Hub-side scenes stay-behind, menu items, subscriptions, and inbox. A vanished client — a killed process, a dropped socket — is reaped 30 minutes after its last activity, so its Hub state cannot leak until restart. Session teardown is isolated: one session ending does not disturb another.

### 6.2 The Loopback Policy

luxd binds loopback by default and refuses a non-loopback `--host` at startup with a clear message, so the bind and the trust policy can never disagree. One `LoopbackTransportPolicy` (`src/punt_lux/transport\_policy.py`) is the single source of the loopback allowlist: it decides which hosts luxd starts on, and it projects the same allowlist into the SDK's transport security settings so the streamable-HTTP transport rejects a foreign `Host` header (DNS-rebinding) or `Origin` header (cross-site). Off-loopback access needs authentication that is not built yet; the multi-machine future in `target.md` arrives together with a bearer token and a bind-derived origin policy.

### 6.3 The Display Socket Is Hub-Internal

The Display listens on a Unix domain socket, and luxd is its only client. Nothing else in the package opens that socket. `DisplayClient` (`src/punt_lux/display\_client.py`) is the class that opens it,

and it is constructed in exactly one place: `ClientRegistry` in `src/punt_lux/domain/hub/clients.py`, which holds luxd's one lazy connection and its reconnect policy.

An AST guard enforces this (`tests/domain/test_display_socket_internal.py`). The guard parses every module in the package and fails if any module outside the allowed set — `src/punt_lux/display\_client.py` and `src/punt_lux/domain/hub/clients.py` — imports or references `DisplayClient`. Detection is by parse tree, not by regex, so every spelling (an aliased import, a multi-line parenthesized import, a whole-module import, a bare or attribute reference) is caught at once, while a mention in a docstring or comment is never mistaken for use. A new reach-around cannot slip back in through a module an enumerated list would have forgotten.

### 6.4 Display Socket Discovery and Auto-Spawn

The Display's socket path resolves in priority order (`src/punt_lux/paths.py`):

1. `$LUX_SOCKET`.
2. `$XDG_RUNTIME_DIR/lux/display.sock`.
3. `/tmp/lux-$USER/display.sock` (fallback, chosen to stay within macOS's ~104-character `AF_UNIX` path limit).

A PID file is written at {`socket_path`}`.pid` on startup, and luxd verifies liveness via `os.kill(pid, 0)` before connecting. `DisplayPaths.ensure()` spawns the Display if it is not running, detached from the caller's process group, and polls socket existence and PID liveness every 100 ms until ready or a 5 s timeout. When a send to the Display fails with an `OSError`, the replicator drops the stale client so the next send reconnects.

### 6.5 The Display Render Loop

The Display runs an ImGui render loop via `hello_imgui.run()`. Each frame, the `DisplayServer._on_frame()` callback runs four non-blocking phases: accept new socket connections, poll the connection for messages and dispatch each by type, render the active scenes and frames, and flush any queued interaction messages back to the Hub. A frame completes in microseconds when idle; `hello_imgui.fps_idling.fps_idle = 30.0` caps the idle rate, and GLFW vsync paces active frames.

Table filtering runs in the render loop with zero Hub round-trips. `TableRenderer` draws the search and combo filter controls, applies AND logic across active filters (case-insensitive substring for search, exact match for combos), resets pagination to page zero on a filter change, and renders a two-column detail grid for the selected row.

## 7 Client Surfaces

Every surface is a thin adapter over the operations facade (§2.2). An MCP tool parses its arguments, calls one operation, and formats the result; it holds no validation, no scene construction, and no display round-trips.

### 7.1 MCP Tools

The display-facing tools live in `src/punt_lux/tools/tools.py` and `src/punt_lux/tools/composite\_tools.py`; the session pub-sub tools live in `src/punt_lux/tools/subscribe\_tools.py`.

| Tool | Operation and purpose |
|---|---|
| **Scenes** | |
| `show` | `render` — install a scene from element dicts. |
| `show_table` | `render_table` — compose a filterable table tree and delegate to `render`. |
| `show_dashboard` | `render_dashboard` — compose a metrics, charts, and table tree and delegate to `render`. |
| `update` | `update` — apply patches by element ID. |

| Tool | Operation and purpose |
|---|---|
| `clear` | `clear` — remove the caller's scenes. |
| **Menus** | |
| `set_menu` | `set_menu` — replace the Hub-owned menu bar. |
| `register_tool` | `register_menu_item` — add a tool item for the caller's session. |
| `list_menus` | `list_menus` — read the menu bar. |
| **Introspection (Hub-authoritative)** | |
| `inspect_scene` | `inspect_scene` — a scene's element tree from the store, with each element's `render_path` (`"abc"` or `"legacy"`) and `resolved_props`. |
| `list_scenes` | `list_scenes` — every live scene and frame from the store. |
| `list_clients` | `list_clients` — the Hub's sessions and their scopes. |
| **Display facts (proxied)** | |
| `get_display_info` | `get_display_info` — backend, geometry, FPS, PID, uptime. |
| `get_window_settings` | `get_window_settings` — opacity, font scale, decoration, idle FPS. |
| `set_window_settings` | `set_window_settings`. |
| `get_theme` | `get_theme` — the active theme and the available themes. |
| `set_theme` | `set_theme`. |
| `set_frame_state` | `set_frame_state` — minimize or restore a frame. |
| `list_recent_events` | `list_recent_events` — the display's recent interactions. |
| `list_errors` | `list_errors` — the display's recent errors. |
| `screenshot` | `screenshot` — refuses with `rejected`: framebuffer capture is unsolved below the message layer (DES-028). |
| `ping` | `ping` — display liveness probe. |
| **Per-repo config** | |
| `display_mode` | `read_display_mode`. |
| `set_display_mode` | `write_display_mode`. |
| **Session pub-sub** | |
| `subscribe` | `subscribe` — subscribe the session to a topic in its own scope. |
| `unsubscribe` | `unsubscribe`. |
| `publish` | `publish` — fan a business event to the session's subscribers. |
| `recv` | `receive` — take the next session-scoped business event. This is not the display interaction stream. |

An introspection tool reads Hub-authoritative state: `inspect_scene`, `list_scenes`, and `list_clients` ask the Hub, not the Display. A display-fact tool — a theme, a window setting, a framebuffer, the display's own ring buffers — proxies a read or write over luxd's one connection. The Hub cannot own an ImGui theme or a GPU backend string, so those stay display facts; the caller still reaches them the same way, through a Hub operation. Menu state is the deliberate exception: a menu is interface the agent submitted, so the Hub owns it and the replicator pushes it like any scene.

### 7.2 REST Routes

The REST surface (`src/punt_lux/rest/`) binds each route body to a request model, calls one operation, and returns the result model; FastAPI derives the OpenAPI schema from the model, so there is no second schema to maintain.

| Method and path | Operation | Result |
|---|---|---|
| `PUT /scenes/{id}` | `render` | `SceneShown \| OpError` |
| `PATCH /scenes/{id}` | `update` | `SceneShown \| OpError` |
| `DELETE /scenes` | `clear` | `Cleared` |
| `GET /scenes` | `list_scenes` | `SceneList` |
| `GET /scenes/{id}` | `inspect_scene` | `SceneInspection \| OpError` |
| `GET /clients` | `list_clients` | `ClientList` |
| `GET /menus` | `list_menus` | `MenuList` |
| `PUT /menus` | `set_menu` | `Ok \| OpError` |
| `POST /menus/items` | `register_menu_item` | `Ok \| OpError` |
| `GET /events` | `list_recent_events` | `RecentEvents` |
| `GET /errors` | `list_errors` | `RecentErrors` |
| `GET /display` | `get_display_info` | `DisplayInfo \| OpError` |
| `GET,PUT /display/theme` | `get/set_theme` | `ThemeState \| OpError` |
| `GET,PATCH /display/window` | `get/set_window_settings` | `WindowSettings \| OpError` |
| `PATCH /display/frames/{id}` | `set_frame_state` | `Ok \| OpError` |
| `GET /display/ping` | `ping` | `Pong \| OpError` |
| `GET,PUT /display-mode` | `read/write_display_mode` | `DisplayModeState \| OpError` |

Pub-sub is not on REST. Publishing is scoped to the publisher's own session, and a REST caller shares one anonymous default scope with no subscribe route, so a REST publish could return success while being structurally unable to reach any subscriber. The pub-sub operations stay MCP-only until a REST caller has a real session identity to subscribe under. For the same reason, every REST call today lands in one default Hub scope; per-caller REST scoping arrives with the multi-user future.

### 7.3 The Command-Line Tool

The CLI is the third thin client of the one engine. `lux show beads` builds the beads element tree with the pure `BeadsBrowser` and sends it to luxd's REST API through `LuxRestClient` (`src/punt_lux/rest\_client.py`); it never touches the display socket. The client locates luxd's port from `HubPaths`, posts the request model, and reads the typed result. Two outcomes are distinct: luxd being unreachable (no port file, a refused connection, a stalled response) raises `HubUnavailableError` with an actionable message, while the Hub's refusal of a reachable request comes back as a typed `OpError` mapped from the HTTP status.

## 8 Performance

### 8.1 Frame Budget

| Parameter | Value | Notes |
|---|---|---|
| Idle FPS | 30 | `fps_idling.fps_idle = 30.0` |
| Active FPS | 60+ | GLFW vsync, GPU-bound |
| Socket poll timeout | 0 | Non-blocking `select()` |
| Max message size | 16 MiB | Hard limit in `FrameReader` |
| Connection timeout | 5.0 s | `ensure()` and handshake |
| Session idle reap | 1800 s | MCP session idle timeout |
| Frame-expiry poll | 1.0 s | `ExpirySweep` idle cap |
| Texture loading | Lazy | First render triggers PIL + OpenGL upload |
| Event ring buffer | 200 entries | Display recent interactions |
| Error ring buffer | 100 entries | Display recent errors |

## 8.2 Critical Path

A write does not wait on the Display. An operation returns as soon as it has updated `HubDisplay` and marked the scene dirty; the replicator does the socket send on its own thread, so a stuck or dead Display never blocks an agent-facing call. This liveness property — no mutator is ever disabled by display I/O — is one of the invariants the replicator formal model checks (Section §11).

The Display's per-frame cost is a `select()` on the one connection, a buffer drain, the ImGui draw calls (linear in visible element count), and an event flush. Table filtering is $O(R \cdot C)$ per frame in rows and columns, fast for tables under ∼10,000 rows, with pagination bounding the rendered rows.

## 8.3 Memory

- **Scene storage.** Scenes are Python dataclass trees in memory, bounded per message by the 16 MiB cap; dismissed scenes are garbage-collected.
- **Texture cache.** OpenGL texture IDs are held until exit with no eviction policy, so image-heavy sessions accumulate GPU memory.
- **Ring buffers.** The event and error buffers are bounded deques; constant memory.

## 8.4 Known Limitations

1. **No ImGui ID scoping.** Widget IDs derive from `element_id` alone, so ImGui internal state can bleed between tabs with colliding IDs.
2. **No texture eviction.** Long sessions with many images accumulate GPU memory.
3. **Per-frame filtering.** Table filters re-run every frame even when nothing changed; harmless at current scale.
4. **No screenshot capture.** Framebuffer capture is unsolved below the message layer, so `screenshot` refuses (DES-028).

# 9 Security

## 9.1 Trust Boundaries

There are two. The first is luxd's HTTP surface: it binds loopback, refuses a non-loopback bind at startup, and rejects a foreign `Host` or `Origin` header (Section §6). The second is the Display's Unix socket: luxd is its only client, filesystem permissions protect the socket directory (user-only, mode `0700`), and an oversize or malformed frame disconnects the peer rather than crashing the Display.

## 9.2 Render Function Consent

A render function is the one element kind that executes arbitrary code, and its security model is consent-based, not sandbox-based. A best-effort AST scan flags suspicious imports and dangerous builtins as a UX signal, not a boundary; a consent modal shows the full source with any warnings, and the user must click Allow to run it; execution is not sandboxed — the code runs in the Display's process. A render function should be treated as user-approved code, equivalent to a script the user chose to run.

## 9.3 Protocol Robustness

- Malformed JSON disconnects the peer; it does not crash the Display.
- An unknown message type deserializes as `UnknownMessage` for forward compatibility.
- An oversize frame is rejected by `FrameReader` and converted to a disconnect.
- A patch that tries to change an element's `id` or `kind` is dropped.
- Double-remove of a client is idempotent.

# 10 Platform Integration

## 10.1 macOS

- The Display runs as a normal Dock app; the Dock icon and Cmd-Tab entry aid operational debugging.
- `setproctitle("Lux")` names the process in `ps` and `top`.
- Font discovery prefers Arial Unicode or Helvetica and merges Apple Symbols and STIX Two Math for mathematical glyphs.
- Socket paths stay within the ∼104-character `AF_UNIX` limit; tests use `tempfile.mkdtemp(prefix="lux-")`.

## 10.2 Linux

- Font discovery uses DejaVu Sans or Noto Sans and merges Noto Sans Symbols2 and Noto Sans Math.
- Socket placement prefers `$XDG_RUNTIME_DIR/lux/`.
- Window managers use the X11 window title ("Lux").

# 11 Formal Models

Four Z specifications hold parts of the system to a checked model. Each is type-checked with Spivey's `fuzz` and model-checked with ProB; the first two model concurrency the current code runs, and the third models the Display singleton lifecycle. The fourth is the legacy display-server model, which the current display code is still refined against.

| Model | What it proves |
|---|---|
| `docs/hub_replicator.tex` | The single-writer, two-lock replication discipline. Six invariants I1–I6: only the replicator writes to the Display (single writer); no mutator is ever blocked by display I/O (agent-path liveness); no thread holds the store lock and the send lock at once (no lock cycle); the display shows the store's latest state at rest, including after a reap and respawn (no lost update); a respawn follows a kill (one display); and the menu bar is repainted before rest (menu durability). |
| `docs/frame_expiry.tex` | The re-show / expiry atomicity invariant: a frame re-armed with a fresh TTL is never torn down by a stale deadline, because both the re-show and the sweep run under the one store lock. The fidelity variant drops the atomicity and ProB reaches the exact bad state where a freshly re-shown frame has been torn down (Section §5). |
| `docs/display_lifecycle.tex` | The Display-singleton spawn lifecycle (DES-037/038): at most one Display exists across concurrent spawn attempts, and the lifecycle is deadlock-free. |
| `docs/architecture/display-server.tex` | The legacy display-server state machine: client connect and disconnect, scene replacement, patching, and event flush. Amended so that an empty-element push is a scene removal, matching the frames-die-with-scenes behavior (Section §5); the refinement harness now drives an empty push and holds the code to the amended model. |

Refinement tests in `tests/test_display_refinement.py` tie the concrete Python back to the legacy model through an abstraction function, and partition tests in `tests/test_display_partition.py` derive cases from its operation schemas. The legacy model predates multi-scene tabs and the Hub/Display split; its abstraction maps multi-scene state to the active tab.

# 12 Test Architecture

The suite is layered; the exact count changes, so this records the shape.

- **Protocol and codec.** Framing, serialization roundtrips, typed draw-command decode, value validation, and self-validation (DES-039). Every element kind has a Level-1 roundtrip; a protocol change without one is unshippable.
- **Operations and surfaces.** The operations facade over fakes, the REST routers via `TestClient`, the MCP adapters, and the CLI's `LuxRestClient`.
- **Hub domain.** `HubDisplay` authority and ownership, the D21 dispatch, the frame lifecycle, and the replicator.
- **Guards.** The display-socket AST guard (§6) and the MCP-tool characterization corpus, which replays a captured snapshot per tool so the string contract cannot drift silently.
- **Formal-model alignment.** The refinement and partition tests against the legacy display-server model.
- **Integration and e2e.** Marker-gated tests exercise the real Hub/Display boundary; the business-event-loop harness (`tests/e2e/`) proves the full bidirectional D21 loop for a composed surface across the faithful in-memory connection.

Quality gates run through the Makefile: `ruff` (lint and format), `mypy` and `pyright` (both strict), `pytest`, `check-oo` (the OO ratchet against `.oo-baseline.json`), and the snapshot-parity gate. `make check` is the composite gate.